IGUIDE FORUM 2026

From Manual GIS Workflows to Reproducible Dam-Failure Risk Analytics

A PostGIS REST API and Dashboard Reference Implementation for Downstream Dam-Failure Vulnerability

Jungha Woo
wooj@purdue.edu

Michael Englert
michael.englert@usu.edu

Anand Padmanabhan
apadmana@illinois.edu

Xiao Liu
liu4201@purdue.edu

Furqan Baig
fbaig@illinois.edu

Courtney Flint
courtney.flint@usu.edu

Erick Li
zhiyuan5@illinois.edu

X. Carol Song
cxsong@purdue.edu

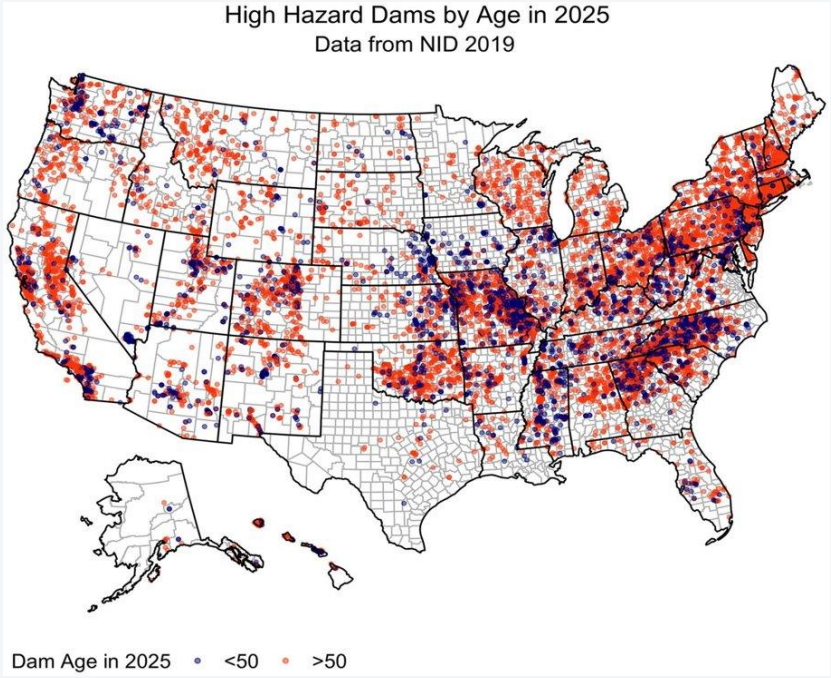
Purdue University (Rosen Center for Advanced Computing) · University of Illinois Urbana-Champaign (CyberGIS Center) · Utah State University

dams.i-guide.io · REST API docs: 149.165.154.170:30080/docs

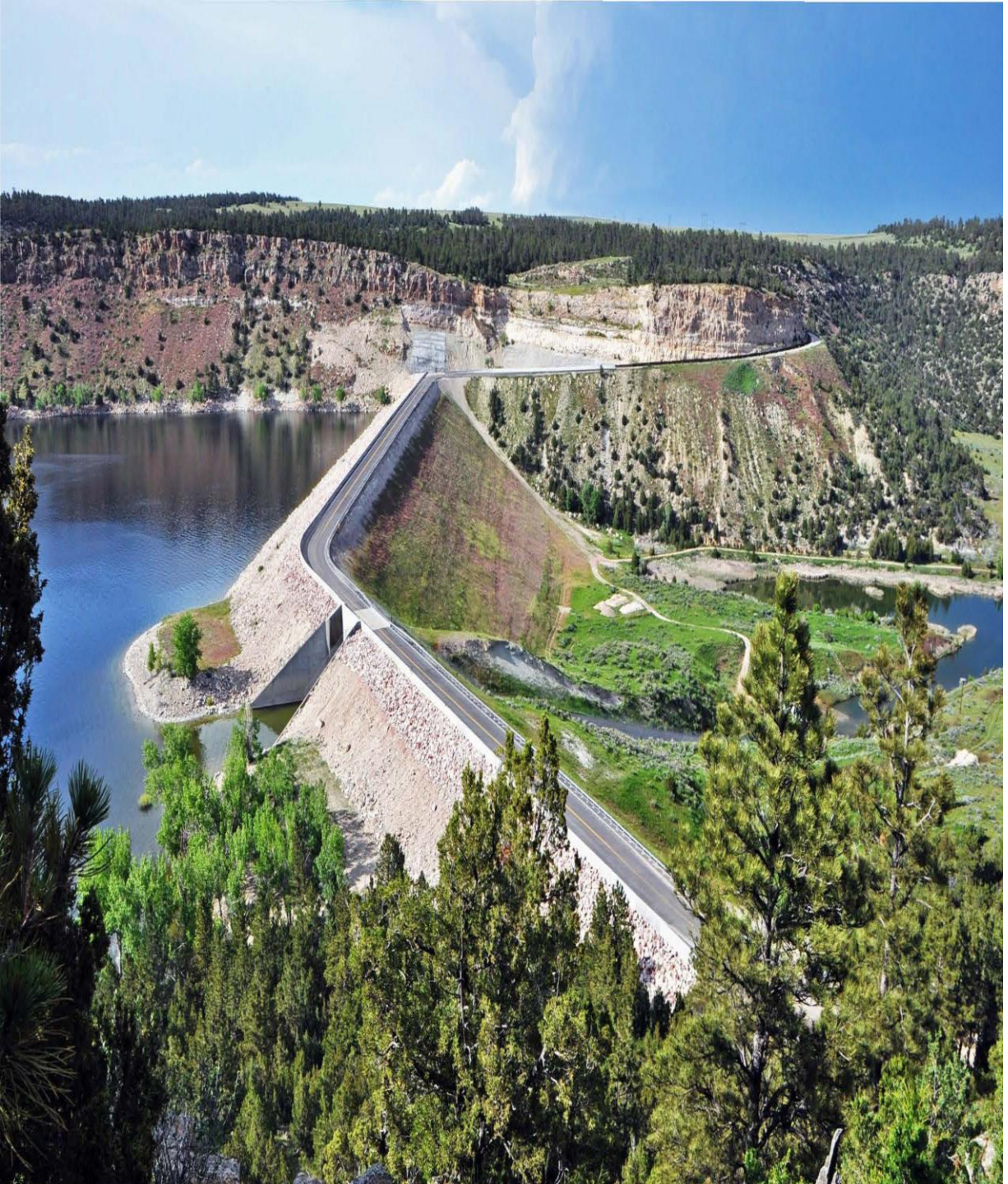
THE PROBLEM

Aging Infrastructure, an Undersized Risk Model

- The US dam inventory is aging, and failure consequences reach well beyond the dam footprint itself.
- Hazard classification (low / significant / high) rests almost entirely on one dimension: potential loss of life.
- Social, ecological, and technological effects are not consistently folded into downstream risk assessment.



92,590 dams in the US National Inventory of Dams (2019 data)



MOTIVATION

The Risk Question

- Downstream risk isn't static — population growth, aging infrastructure, and shifting weather all change the picture over time.
- Overtopping — water spilling over the top of a dam — causes about one-third of all US dam failures.
- Drought concerns push operators to keep reservoirs fuller, which raises the odds of overtopping when a storm does hit.
- That's the real point: risk assessment can't be a one-time snapshot. It has to be something you can rerun as conditions change.

Hwang, J. & Lall, U. (2024). *Increasing dam failure risk in the USA due to compound rainfall clusters as climate changes. npj Natural Hazards.*

CURRENT PRACTICE

Desktop GIS Gets the Science Right — It Just Doesn't Scale

Expert-led desktop GIS workflows establish sound spatial logic and validated risk assessments. As a system for repeated, timely reassessment, they hit three walls:

Hard to Repeat

The exact steps live in an analyst's head and mouse clicks — not written down in a form a computer can rerun.

Hard to Scale

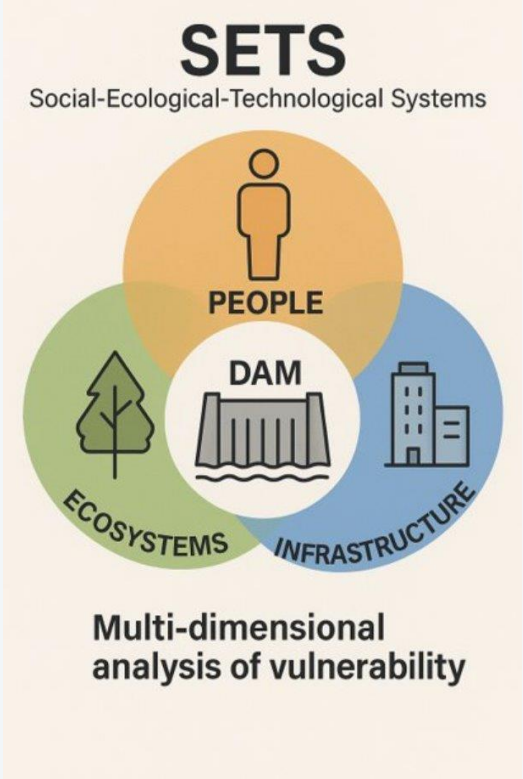
Adding one new factor, or updating after new data arrives, means redoing much of the work by hand.

Hard to Share

Results ship as static files instead of a live service other tools and pipelines can query.

FRAMING

Two Ideas Anchor the Design



A dam's failure puts people, ecosystems, and infrastructure at risk together — not one at a time.

FAIR Data Principles

Findable: Discoverable through the project's public catalog

Accessible: Reachable through an ordinary web dashboard — no special software

Interoperable: Any program can request the same data the same way, over REST

Reusable: Demonstrated end-to-end via public tutorial notebooks on the I-GUIDE Platform

CONTRIBUTION

One Request, the Full Picture

A reproducible pipeline that turns validated desktop mapping work into two connected, open tools:

A REST API

Precomputed, queryable dam-vulnerability data, served over stateless HTTP.

A Dashboard

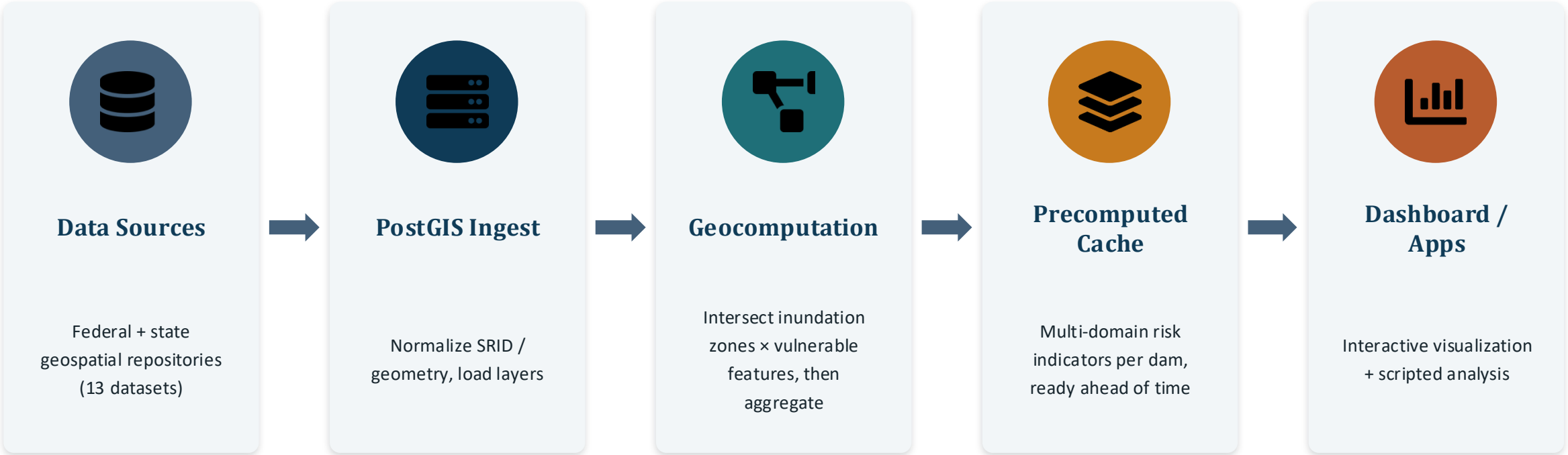
A public, interactive reference client built on top of that same API.

One request, a complete answer

A single API call replaces many manual spatial joins — and returns one combined answer.

SYSTEM OVERVIEW

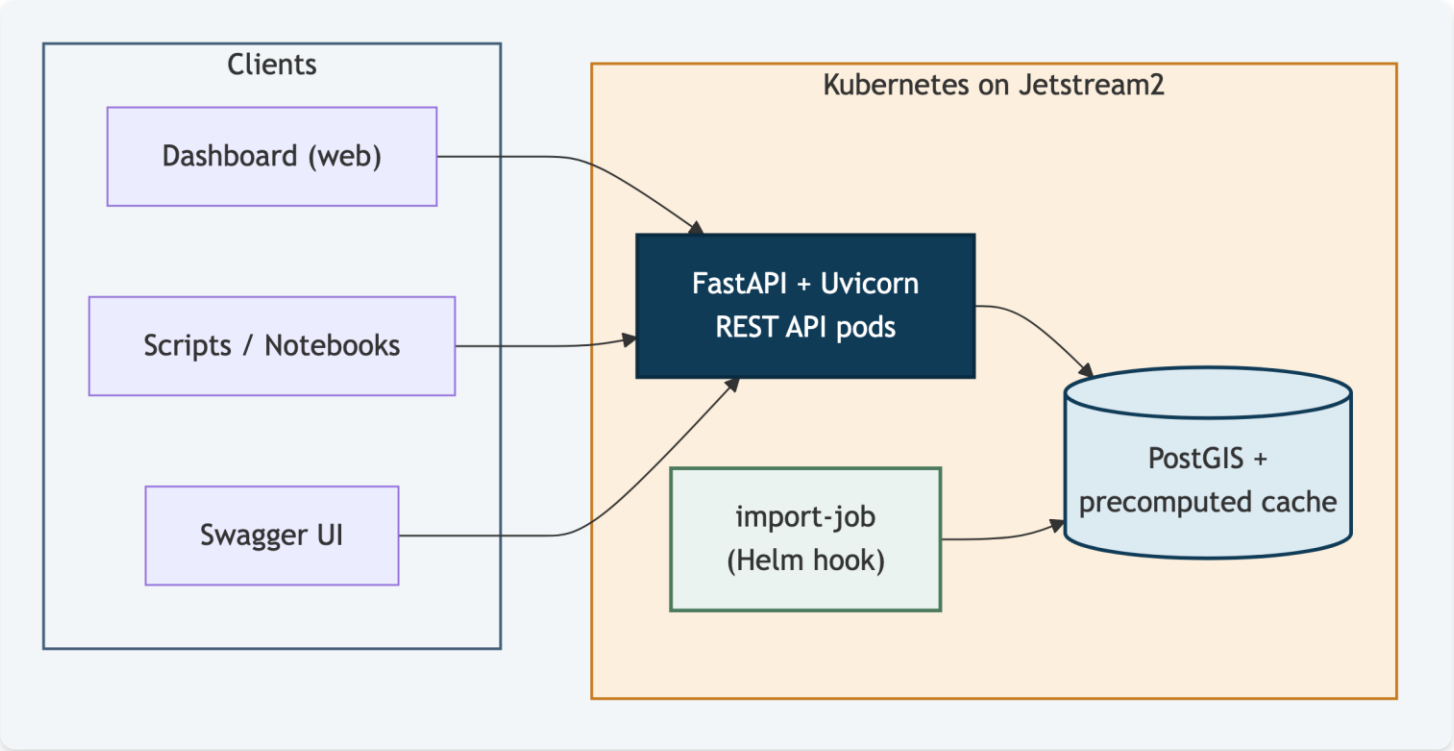
The Data-to-API Pipeline



Import & refresh jobs can be rerun end-to-end to regenerate derived tables under controlled, versioned transformations.

UNDER THE HOOD

Backend Architecture: FastAPI + PostGIS



Clients on the left, the FastAPI service and database on the right — all running as Kubernetes pods on Jetstream2.

Why FastAPI

- Auto-generates the OpenAPI/Swagger docs straight from the route and Pydantic model definitions.
- Pydantic models validate every request before it ever touches PostGIS — bad input fails fast, with structured errors.
- Async, production-grade — served by Uvicorn, confirmed live (server: uvicorn on every response).

Why EPSG:4326? Interoperability Isn't Optional

The platform uses a geography-based strategy on EPSG:4326 (WGS84 geodetic) to keep cross-region interoperability and avoid projection fragmentation across jurisdictions.

Not a mapping convenience — a methodological requirement. Inundation footprints are local, but downstream consequences propagate through interstate systems (e.g., electric transmission dependencies). A common geodetic CRS is what keeps overlay operations and risk indicators comparable across state lines.

INGESTION LOGIC

From Many Inundation Zones to One Footprint per Dam

1

Normalize

SRID & geometry aligned to EPSG:4326 at ingest

2

Select Largest

When a dam has multiple inundation zones, the largest becomes its canonical footprint

3

Load 8 Layers

Transportation, roads, hospitals, WWTP, power transmission, hazardous waste, SVI tracts, protected areas

4

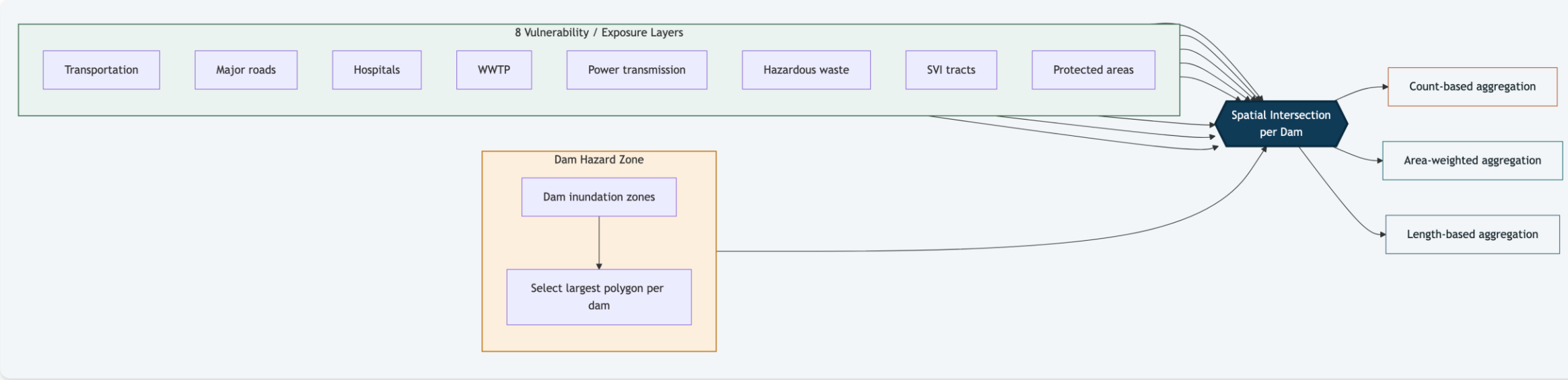
Intersect per Dam

One spatial-intersection query set per dam, against every vulnerability layer

TUTORIAL DEEP-DIVE

Where the Datasets Actually Meet

The same logic as the previous slide, drawn the way it's used in the hands-on tutorial — every layer converges on one spatial intersection, per dam.



8 vulnerability / exposure layers + the dam's own worst-case flood footprint, both feeding the same intersection step, then splitting into 3 aggregation families.

AGGREGATION SEMANTICS

Three Rules, Chosen to Match the Geometry

Count-Based

hospitals · WWTPs · hazardous-waste sites

Simple count of features whose geometry intersects the inundation footprint.

Area-Weighted

population · SVI score · % protected status-1

Each intersecting polygon contributes in proportion to the share of its area inside the footprint — no double counting.

Length-Based

major-road length · transmission-line length

Summed length of linear features (roads, pipelines, transmission lines) crossing into the footprint.

One nuance worth flagging: hospital trauma level is aggregated as the max value present — but level 1 is the highest acuity and level 5 the lowest, an inverted scale relative to most of the schema.

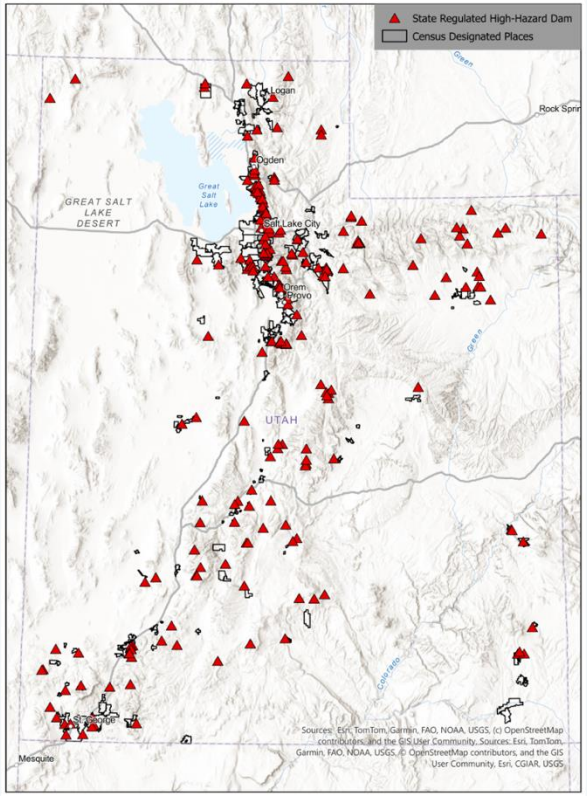
CASE STUDY

227 Dams in Utah

The reference deployment: 227 state-regulated, high-hazard dams in Utah, built with the Utah Office of Dam Safety.

It's the concrete proof-of-concept behind everything in this talk — but the architecture itself isn't Utah- or dam-specific (more on that shortly).

Collaborator: Utah Office of Dam Safety



227 state-regulated high-hazard dams plotted across Utah

OPERATIONS

Reproducibility Is an Engineering Property

Swap source file

Researcher replaces a dataset in the shared /data volume

helm upgrade

One standard command — no server code or SQL to touch

import-job fires

Declared as a Helm post-install / post-upgrade hook — runs automatically

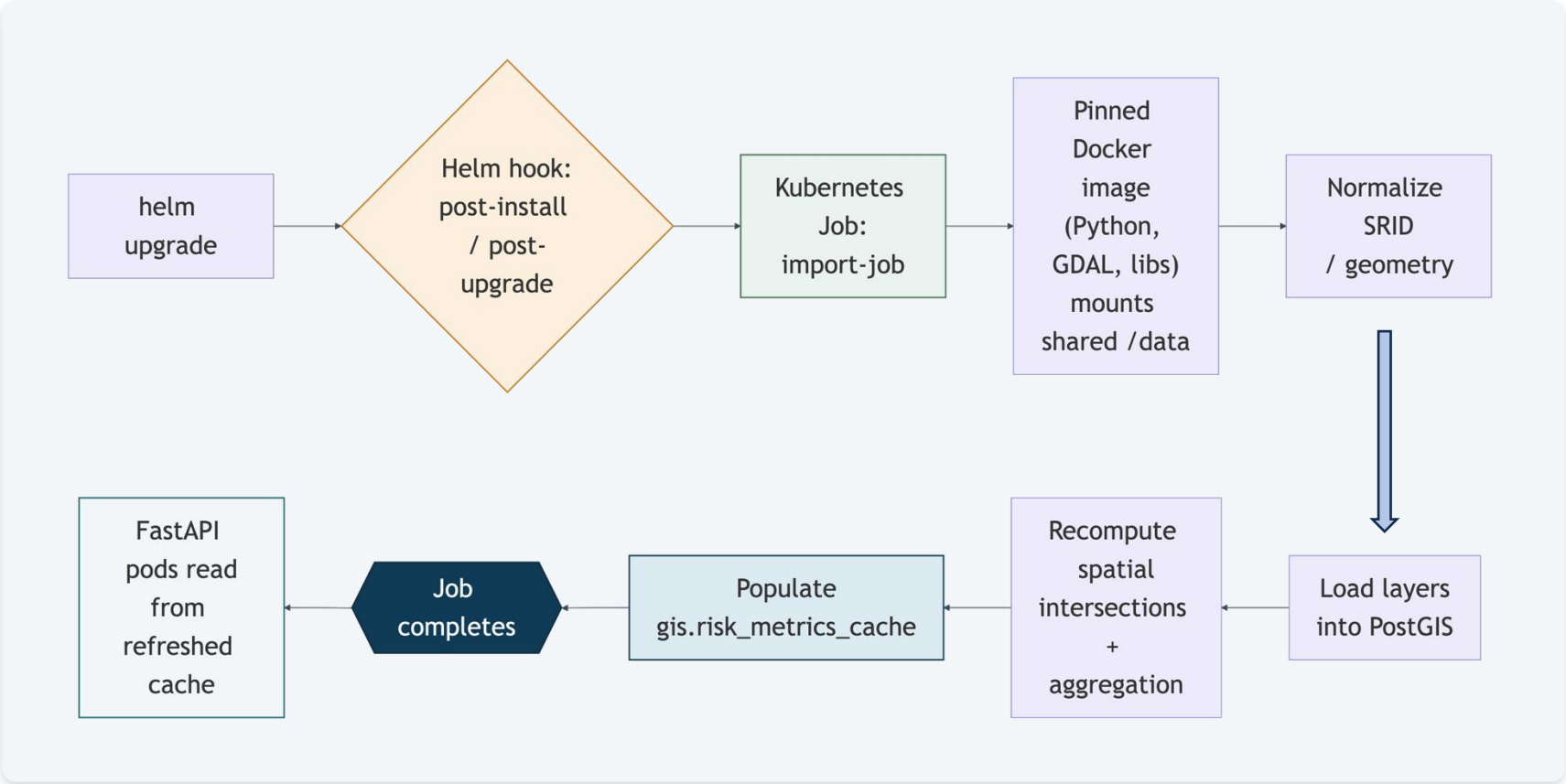
Cache refreshed

All layers reloaded, all indicators recomputed, cache tables updated

Pinned environments end silent drift. The Docker image fixes Python packages, GDAL version, and system libraries for the import environment — so dev, test, and production run the identical orchestration logic (schema, job triggers, secrets) with no undocumented version skew.

What Actually Happens on helm upgrade

The import-job.yaml Helm hook, end to end. Read left to right, then down, then right to left.



This is the whole point of the hook: *helm upgrade is the entire researcher-facing interface for refreshing every dam's numbers — no SQL, no server access.*

CYBERINFRASTRUCTURE

Where This Runs, and How It Connects to I-GUIDE

Where It's Deployed

The REST API and dashboard run on ACCESS Jetstream2 — NSF-funded research-cloud VMs — not directly on the I-GUIDE Platform itself.

The I-GUIDE Platform's own infrastructure is built for hosting Jupyter notebooks and courses. This service needed its own always-on, containerized deployment — so Kubernetes on Jetstream2 was the better fit.

Bridging to I-GUIDE

Four Jupyter notebooks built for the I-GUIDE Forum tutorial session, hosted on the I-GUIDE Platform — walking through the backend, the frontend, and how one helm upgrade refreshes the underlying datasets.

`platform.i-guide.io/notebooks/f711f98c-…`

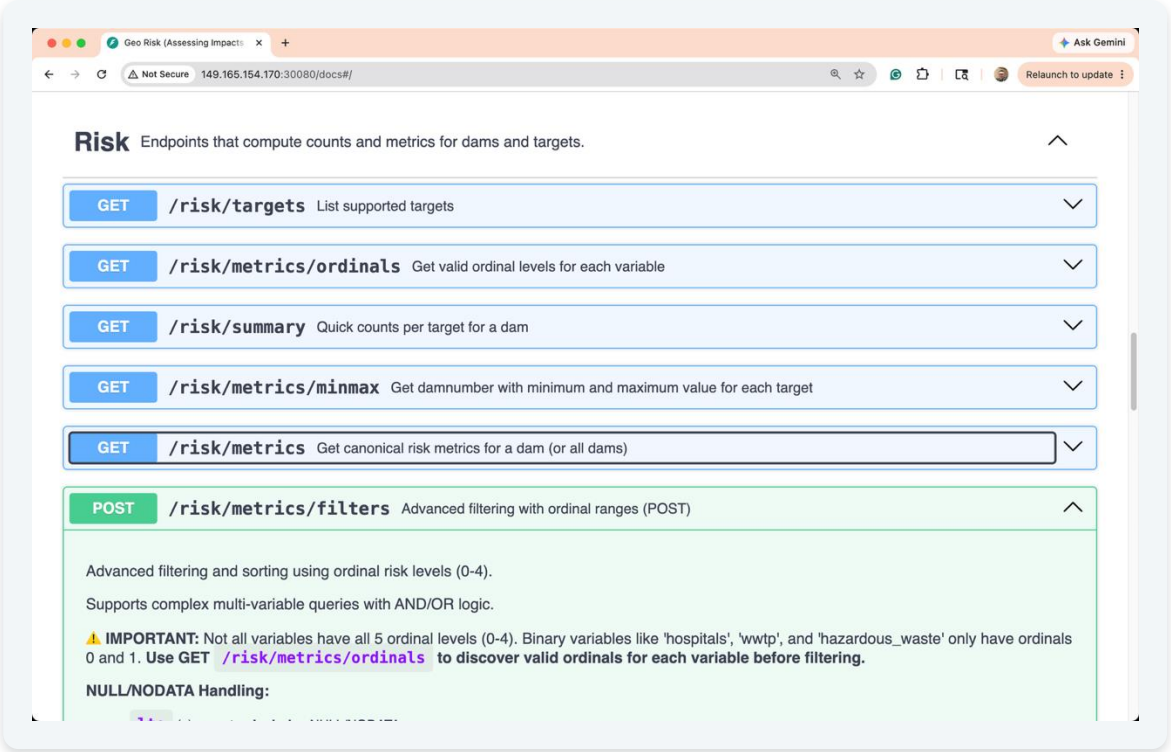
`github.com/junghawoo/I-GUIDE-forum-tutorial`

(public — code + notebooks)

THE LIVE SYSTEM

A System You Can Ask Questions Of

Every dam's indicators are precomputed into a cache table ahead of time. On-the-fly recomputation across all dams and layers takes 15+ minutes; a cached lookup returns in under 100ms — and any program, not just the dashboard, can ask for it directly, over a documented REST contract.



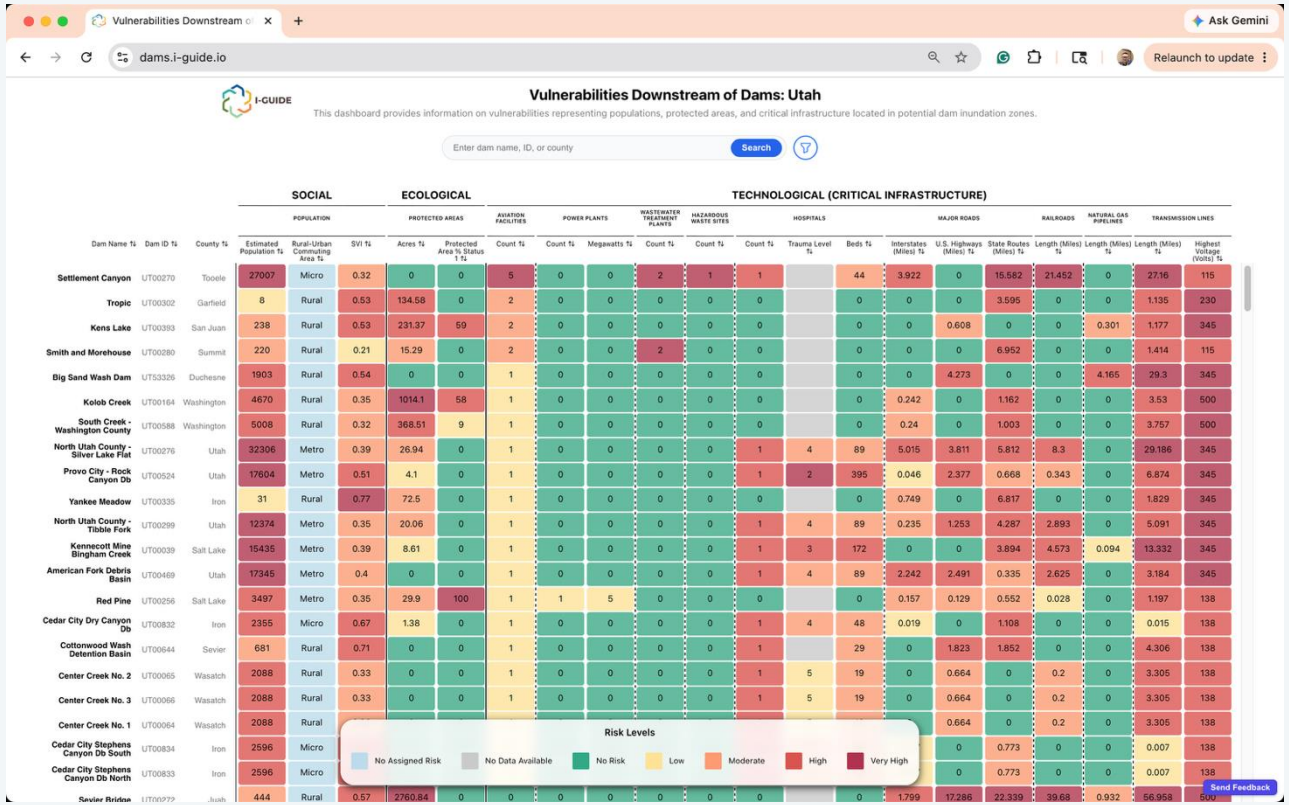
The real, live Swagger interface — self-documenting from the OpenAPI spec.

What this makes possible

- A researcher, a script, or the dashboard all get the exact same answer for the exact same query.
- Queries stay index-friendly (sargable): SRIDs are normalized at ingest so PostGIS uses GiST spatial indexes instead of full scans.
- Every endpoint and its parameters are documented, so results are easy to check and reuse.

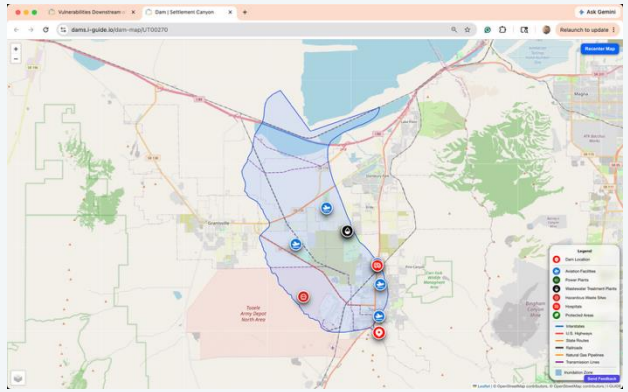
REFERENCE CLIENT

The Dashboard: A Window Into the Numbers



dams.i-guide.io — live data for Utah's high-hazard dams

Filter — e.g. by hospital-bed count **Search** — by dam name or ID



Map view — a real dam's flood zone and nearby infrastructure

These risk levels are our own documented cut-offs — a starting point we defined, not an absolute standard. The dashboard just displays them consistently.

LOOKING AHEAD

The Same Pattern, a Different Hazard

Hazard-Specific

△ The hazard-zone source (dam inundation maps from Utah SGID)

△ Dam-specific attributes from the National Inventory of Dams

Reusable, As-Is

✓ The ingestion pipeline: normalize to one CRS, pick the worst-case footprint

✓ The three aggregation rules (count / area-weighted / length-based)

✓ The REST API contract (metrics / ordinals / filters)

✓ The dashboard reference client and the Helm/Kubernetes deployment pattern

Illustrative, not yet built: swap the dam inundation zone for a river flood-extent map, and the rest of the pipeline — overlay logic, aggregation, API, dashboard — carries over largely unchanged.

SCOPE

What This Platform Is — and Isn't

Checked

- ✓ Data loads correctly and completely
- ✓ Each output table follows its expected structure
- ✓ Computed numbers pass basic sanity checks
- ✓ The same request always returns the same answer
- ✓ Every step is scripted, not done by hand

Not Yet / Out of Scope

- △ Doesn't model how flooding physically happens — it uses flood maps that other agencies already produced
- △ Hasn't yet been stress-tested at a larger, multi-state scale
- △ The Low / Moderate / High / Very High cut-offs for most indicators were set by our own team (e.g., log scales, quartile splits) — a documented starting point, not an absolute standard. The one exception: a dam's own hazard rating comes from the state or federal source.

TRY IT YOURSELF

Live and Available Now

REST API Docs (Swagger)

`149.165.154.170:30080/docs`

Dashboard

`dams.i-guide.io`

Tutorial Notebooks + Code (public)

`https://tinyurl.com/aging-dam-rest-api`

Slides (PDF)

Scan to download the exported PDF

The core aging-dams-workflow repository is currently private; the tutorial repo above is public and walks through the same patterns end-to-end.

CONCLUSION

A Reusable Pattern, Demonstrated on Dams

- A reproducible, containerized pipeline replaces ad hoc desktop mapping — for dams, and for other hazard domains built the same way.
- Precomputing the answers turns a 15-minute spatial join into an instant, scriptable REST query anyone — or any program — can call.
- Ingest → normalize → aggregate → serve → visualize is the reusable part; the dam data is just one instance of it.

Thank you — discussion welcome

dams.i-guide.io · API docs: 149.165.154.170:30080/docs · github.com/junghawoo/I-GUIDE-forum-tutorial · wooj@purdue.edu